

```

struct ethhdr {
    unsigned char  h_dest[ETH_ALEN]; /* destination eth addr */
    unsigned char  h_source[ETH_ALEN]; /* source ether addr */
    __be16         h_proto; /* packet type ID field */
    __attribute__((packed));
};

```

Figure 5: Structure of the Ethernet header

```

struct iphdr {
    #if defined(_LITTLE_ENDIAN_BITFIELD)
        __u8   ihl:4,
              version:4;
    #elif defined(_BIG_ENDIAN_BITFIELD)
        __u8   version:4,
              ihl:4;
    #else
        #error "Please fix <asm/byteorder.h>"
    #endif

    __u8   tos;
    __be16 tot_len;
    __be16 id;
    __be16 frag_off;
    __u8   ttl;
    __u8   protocol;
    __sum16 check;
    __be32 saddr;
    __be32 daddr;
    /*The options start here. */
};

```

Figure 6: Structure of the IP header

Use `fflush` to avoid the input-output buffer problem when writing into a file.

Extracting the IP header

The IP layer gives various pieces of information like the source and destination IP address, the transport layer protocol, etc. The structure of the IP header is defined in the `ip.h` header file (see Figure 6).

Now, to get this information, you need to increment your buffer pointer by the size of the Ethernet header because the IP header comes after the Ethernet header:

```

unsigned short iphdrlen;
struct iphdr *ip = (struct iphdr*)(buffer + sizeof(struct ethhdr));
memset(&source, 0, sizeof(source));
source.sin_addr.s_addr = ip->saddr;
memset(&dest, 0, sizeof(dest));
dest.sin_addr.s_addr = ip->daddr;

fprintf(log_txt, "\t|-Version          : %d\n", (unsigned int)ip->version);

fprintf(log_txt, "\t|-Internet Header Length : %d DWORDS or %d Bytes\n", (unsigned int)ip->ihl, ((unsigned int)(ip->ihl))*4);

```

```

fprintf(log_txt, "\t|-Type Of Service   : %d\n", (unsigned int)ip->tos);

fprintf(log_txt, "\t|-Total Length       : %d Bytes\n", ntohs(ip->tot_len));

fprintf(log_txt, "\t|-Identification   : %d\n", ntohs(ip->id));

fprintf(log_txt, "\t|-Time To Live       : %d\n", (unsigned int)ip->ttl);

fprintf(log_txt, "\t|-Protocol          : %d\n", (unsigned int)ip->protocol);

fprintf(log_txt, "\t|-Header Checksum    : %d\n", ntohs(ip->check));

fprintf(log_txt, "\t|-Source IP          : %s\n", inet_ntoa(source.sin_addr));

fprintf(log_txt, "\t|-Destination IP     : %s\n", inet_ntoa(dest.sin_addr));

```

The transport layer header

There are various transport layer protocols. Since the underlying header was the IP header, we have various IP or Internet protocols. You can see these protocols in the `/etc/protocols` file. The TCP and UDP protocol structures are defined in `tcp.h` and `udp.h` respectively. These structures provide the port number of the source and destination. With the help of the port number, the system gives data to a particular application (see Figures 7 and 8).

The size of the IP header varies from 20 bytes to 60 bytes. We can calculate this from the IP header field or IHL. IHL means Internet Header Length (IHL), which is the number of 32-bit words in the header. So we have to multiply the IHL by 4 to get the size of the header in bytes:

```

struct iphdr *ip = (struct iphdr *) (buffer + sizeof(struct ethhdr));
/* getting actual size of IP header */
iphdrlen = ip->ihl*4;
/* getting pointer to udp header */
struct tcphdr *udphdr = (struct udphdr *) (buffer + iphdrlen + sizeof(struct ethhdr));

```

We now have the pointer to the UDP header. So let's check some of its fields.



Note: If your machine is little *endian*, you have to use `ntohs` because the network uses the big *endian* scheme.